

太原理工大学先进计算机系统实验室 (ACSL)

寒假研学第一次学习路线

学习情况：掌握了数字电路的基本知识，并且搭建了简单的计数器和电子时钟。

学习目标：整个寒假研学期间我们将进行**数字电路，计算机组成原理，数字设计，数据结构，C语言五部分学习**，而本周我们需要**搭建一个最简单的处理器**，而我们现在掌握的这些知识显然是不够的。想要做处理器，先要明白，处理器是用来做什么的，计算机的架构是怎样的，处理器在计算机架构中承担什么角色。也因此我们本周的学习中会来阐释一些和处理器相关的重要概念，以及亲手搭建一个简单的处理器，并为下一周我们完成一个基于riscv32e指令集的带有图片展示效果的处理器奠定基础。

计算机组成原理与体系结构

在搭建处理器之前，我们需要先问自己，什么是处理器？处理器在计算机中承担是什么样的角色？也因此我们需要补充一部分的计算机组成原理与体系结构方面的知识。

计算机组成原理，是我们后续实践的基础科目。很难而且不好理解。体系结构，简单来讲就是计算机系统设计的观念与架构，描述计算机在实践中的设计原则。

这两门学科无论是对于之后考研还是工作都会有很大帮助，同时更是我们后续实践的基础。

计算机组成原理与体系结构

计算机组成原理与体系结构

阅读上方文档，记得随手typora记录笔记和继续自己的学习记录，最后提交你的**Markdown**文档（我们推荐大家进行**总结归纳和心得记录**，这真的很有帮助）。

将其中的**蓝框必答题**汇总成一个Markdown文档放入作业提交文件夹中。

支持数列求和的简单处理器

！ 注意事项

- 理解**计算机组成原理与体系结构**对下面的电路搭建至关重要，理解了这方面的知识，后续的相关电路搭建工作都会变得非常简单。

- 如果你觉得自己已经完全理解并掌握各元件的内部构造了，那么就可以直接使用Loigisim自带的元件，但本章刚刚学习的RAM，ROM除外。
- 善用于电路模块，会使你的电路更加清晰，也更容易debug好看很多。
- 如果你对于很多细节部分的知识依旧有不清楚的，请积极**STFW RTFM**。
- 非电路搭建部分的问题需要你汇总成一份文档，最后也需要提交。

通过ISA和状态机模型，我们已经了解程序如何执行。此外，我们也已经学习了数字逻辑电路的基础知识。现在是时候用数字电路实现一个CPU了！

在此之前，我们需要再次明确需要实现的指令集sISA的细节。和上一小节相比，此处还约定了一些寄存器的位宽：

- PC位宽为4位，初值为 0。
- GPR有4个，位宽均为8位。
- 支持如下3条指令。

代码块

```

1      7  6  5  4  3  2  1  0
2  +---+---+---+---+
3  | 00 | rd | rs1 | rs2 | R[rd]=R[rs1]+R[rs2]      add指令，寄存器相加
4  +---+---+---+---+
5  | 10 | rd |   imm   | R[rd]=imm                  li指令，装入立即数，高位补0
6  +---+---+---+---+
7  | 11 |  addr  | rs2 | if (R[0]!=R[rs2]) PC=addr bner0指令，若不等于R[0]则跳转
8  +---+---+---+---+
```

我们将这个用数字电路实现的sISA指令集的CPU称为sCPU。要实现sCPU，我们需要用数字电路实现sISA中的每一个概念。为了简单起见，我们先从最简单的 **li** 指令开始考虑，也即，先实现一个只支持 **li** 指令的sCPU。

只有一条指令的sCPU

我们在学习指令的时候知道了处理器在处理指令的时候是有指令周期的概念，分为取指，译码，执行，写回，更新PC等步骤。

既然需要搭建处理器，那我们的目标就是用数字电路实现上述过程的每一个步骤。

? tips

根据指令周期的不同阶段来设计模块，最后信号的传递直接用各个模块的接口对接即可。

取指 (Instruction Fetch, IF)

首先需要PC寄存器，你之前已经用数字电路实现过寄存器了，因此PC寄存器并不难实现。然后需要根据PC在存储器中找到一条指令，因此我们还需要实现存储器。

存储器和寄存器都可以存储信息，但存储器还支持寻址(addressing)，也即，存储器中的内容按顺序进行排布的。

? 记得C语言的指针吗

当时我们说C语言的指针存储的是地址，那和这里的地址又有什么关系呢？

给出一个地址，存储器可以读出该地址对应的内容，我们可以将存储器看成一个由比特构成的矩阵，矩阵的每一行称为一个存储字(word)，地址就是行的编号，行的数量也称为存储器的深度(depth)。通常一个存储字包含多位数据，其位宽称为存储器的宽度(width)。通常用 深度×宽度 表示一个存储器的规格。例如，一个 2×3 的存储器排布如下，其中 $b_{(x,y)}$ 表示第 x 行第 y 列所存储的比特：

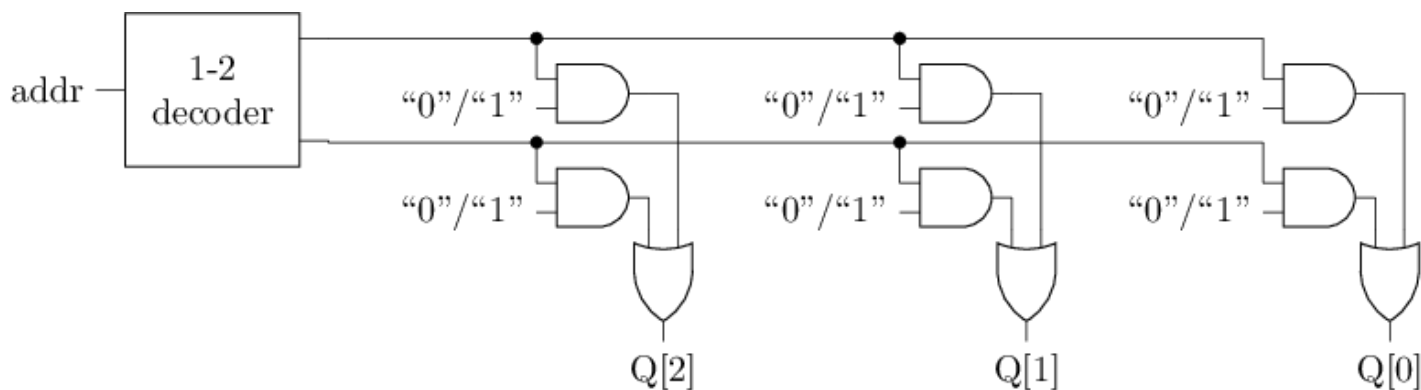
地址	存储字
0	$b_{(0,2)}b_{(0,1)}b_{(0,0)}$
1	$b_{(1,2)}b_{(1,1)}b_{(1,0)}$

从功能上划分，存储器可以分别只读存储器(Read-Only Memory, ROM)和随机访问存储器(Random Access Memory, RAM)，前者不支持写入，而后者支持。对于sISA来说，因为3条指令都不会访问存储器，只有取指操作需要从存储器中读出指令，因此这里可以采用ROM。

一个 2×3 的ROM的结构如下图所示。左上方的译码器又称"地址译码器"。

和地址译码器输出相连的导线称为"字线"(word line)；字线会与相应存储字中的存储单元进行与操作，再对同一列与门的结果进行或操作。

和或门输出相连的导线称为"位线"(bit line)。每条位线对应存储字的一位。



给定地址 `addr`，可以读出ROM中的相应存储字，其工作过程如下。地址译码器将输入的地址转换成一组独热码，由于独热码中只有一位有效，故所有字线中，只有地址 `addr` 对应的字线有效，使得该行中存放的信息可以通过与门。其余行因字线无效，存放的信息均被与门过滤为 `0`。被选中的存储字的每一位经过或门传输到位线，向存储器外部输出。

实际上，图中的地址译码器，与门和或门，在功能上共同构成了一个3位的2选1多路选择器。因此ROM的读操作也可以看作是**从多个存储字中选择一个**，地址 `addr` 就是多路选择器的选择端，所有存储字分别作为多路选择器的数据端。特别地，ROM中存储的信息是直接通过高低电平编码的，因此ROM从功能上也可以看作是数据端为常数的多路选择器。

实现取指功能

通过多路选择器实现一个ROM，并在其中存放数列求和的指令序列（如下），然后通过PC寄存器取出指令。你需要根据你的理解来确定ROM的规格。

代码块

```

1  10001010    # 0: li r0, 10
2  10010000    # 1: li r1, 0
3  10100000    # 2: li r2, 0
4  10110001    # 3: li r3, 1
5  00010111    # 4: add r1, r1, r3
6  00101001    # 5: add r2, r2, r1
7  11010001    # 6: bner0 r1, 4
8  11011111    # 7: bner0 r3, 7

```

译码（Instruction Decode, ID）

译码的目的是**根据指令的编码识别指令的功能**。通过识别出指令的二进制表示(如 `10001010`)，然后查阅指令的功能说明来了解其功能，本质上来说都是属于人工进行译码操作。但现在我们需要用电路来实现这一操作，注意电路中存放的指令是其二进制表示，因此我们需要在二进制层面对指令进行译码。

译码的工作进一步分为操作码译码和操作数译码。

前者是根据指令的操作码来识别指令的功能，例如识别出指令高2位为10，执行 `li` 指令。

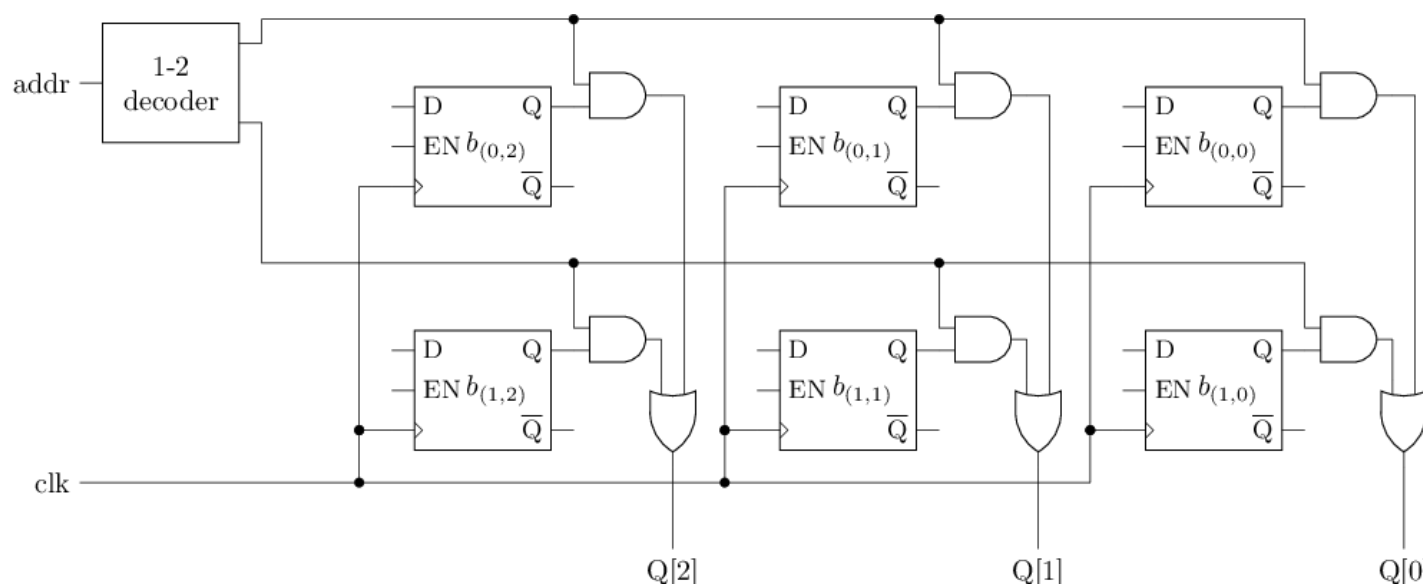
后者是从指令的编码中识别出相应的操作数。对于操作码译码，由于目前只需要实现一条 `li` 指令，因此我们可以认为取到的指令就是 `li`，不存在其他情况。对于操作数译码，我们只需要解析出 `li` 指令中的 `rd` 和 `imm` 字段即可。从电路的角度上看，这只是一些位抽取操作，并不难实现。

执行 (Execute, EX)

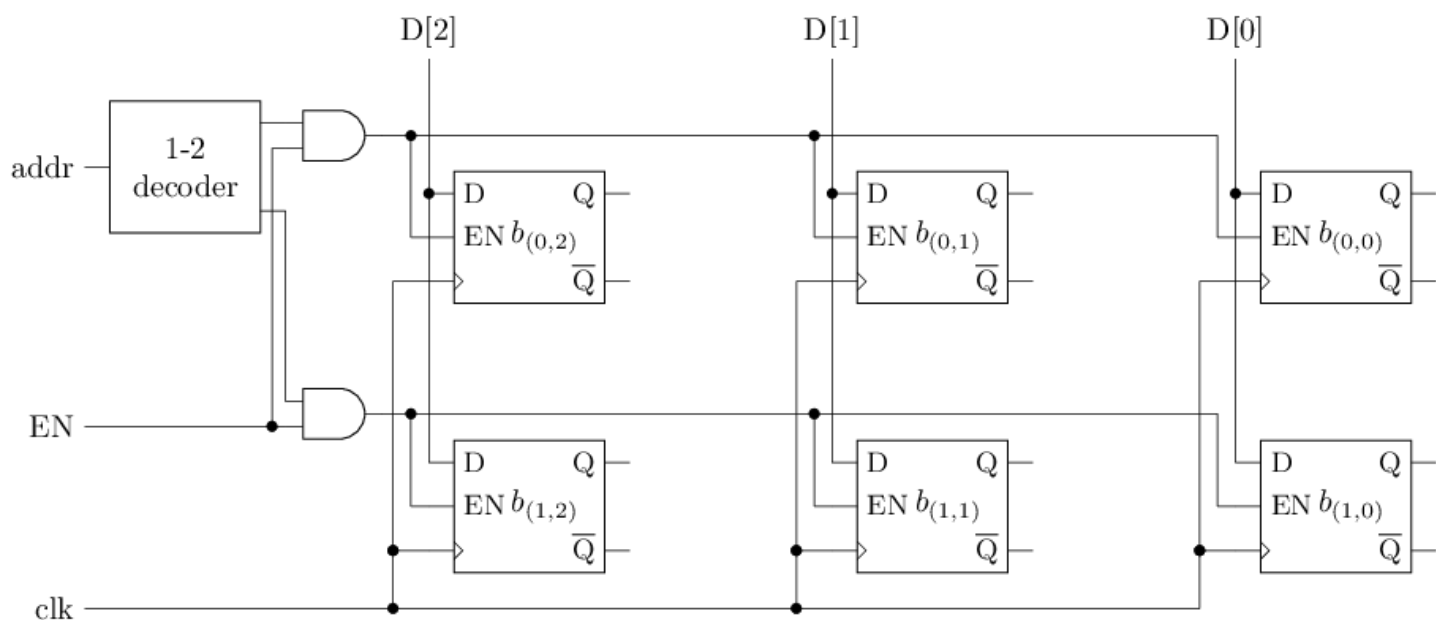
`li` 指令的功能是将立即数 `imm` 写入 `rd` 寄存器中，因此我们需要考虑如何实现ISA的GPR。

GPR通常包含多个寄存器，一次访问通常只访问其中的几个寄存器，因此GPR也应该支持寻址。可以看到，GPR的电路本质也是一个存储器。不过GPR需要作为目的寄存器被指令写入，因此GPR是一个支持写入的存储器，即RAM。

考虑一个 `2x3` 的RAM，其读操作涉及的结构如下图所示。可以看到，除了存储单元采用了**D触发器**之外，其余结构与ROM基本一致，因此RAM的读出操作的工作过程可参考上文ROM相关的部分。

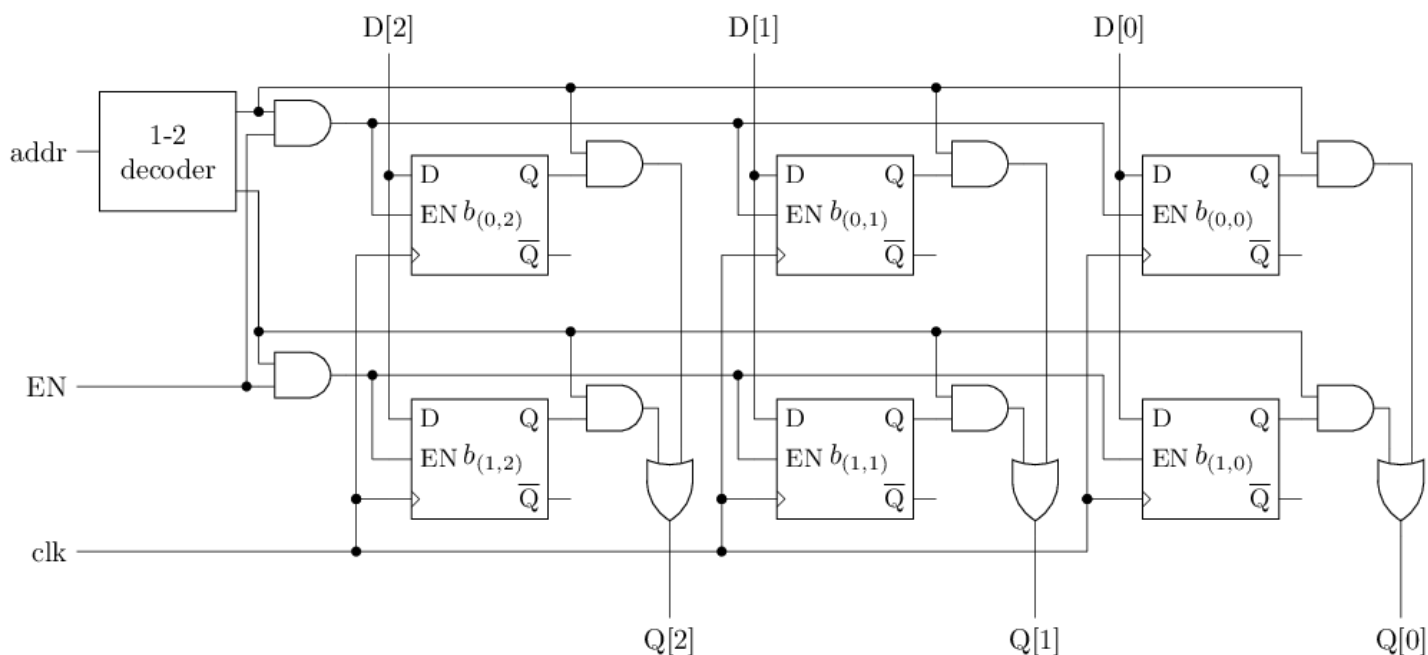


至于RAM的写操作，涉及的结构如下图所示。进行写操作除了需要提供地址外，还需要提供待写入数据 `D`。对于RAM来说，并非所有时刻都需要进行写入操作，例如，在执行 `bner0` 指令时，就不需要写入GPR，因此还需要一个写使能信号 `EN`，指示当前是否需要写入。



给定地址 `addr` 和待写入数据 `D`，在写使能 `EN` 有效时，可以将数据 `D` 写入RAM中的相应存储字，其工作过程如下。待写入数据 `D` 通过位线将每一位分别连接到每一个存储字中相应位的 `D` 端，写使能 `EN` 通过与门对地址译码器输出的独热码进行过滤，并与字线连接，因此在 `EN` 有效的情况下，只有地址 `addr` 对应的字线有效，其余行因字线无效。将每一行的字线分别连接到对应存储器中相应的 `EN` 端，被选中的存储字将更新为待写入数据 `D`，从而完成写入操作。

将读操作和写操作的结构合并，得到RAM的结构如下图所示。这种RAM在同一时刻只能通过一个地址访问其中的一个存储字，称为"单端口RAM"(single port RAM)。相应地，有的RAM在同一时刻可以通过多个地址访问其中的多个存储字，称为"多端口RAM"(multi-port RAM)。可见，增加RAM的"端口"，需要添加额外的电路逻辑，从而使得多个端口的电路逻辑可以同时工作。



和ROM类似，图中的地址译码器，读操作通路上的与门和或门，在功能上共同构成了一个3位的2选1多路选择器。因此对于RAM的读取功能，也可以看作是数据端为寄存器的多路选择器，但写入功能需要在寄存器的基础上额外实现。

实现GPR及其写入功能

搭建一个RAM，从而实现GPR的写入功能。你需要根据你的理解来确定RAM的规格。

！ 注意

虽然我们这里用RAM实现了GPR，但其实RAM和GPR在计算机中是不同层次，不同功能，不同性能的两种存储器。

举个栗子，做一道菜（执行一个程序）

1. RAM 就像厨房里的大冰箱和储物柜。

- 里面存放着所有可能用到的食材（程序和数据）。
- 需要时，你得走过去打开柜门取用，速度相对慢一些。
- 容量很大，可以放很多东西。

2. GPR 就像你手边的砧板和几个小碗。

- 你从冰箱（RAM）里拿出今天要用的少量食材（当前计算所需的数据），放在案板和小碗里。
- 你直接在案板和小碗上进行切菜、搅拌等操作（CPU的运算）。
- 空间非常有限（只能放正在处理的食材），但触手可及，速度极快。

而我们后续无论是电路搭建还是敲代码，对于RAM和GPR往往都是分开考虑的。

更新PC

更新PC的操作十分简单，对PC寄存器加1即可，这可以通过数字电路中的计数器来实现。

实现仅支持li指令的sCPU

根据上文，用数字电路实现 `li` 的指令周期涉及各个部件，并将它们连接起来。实现后，尝试让sCPU执行数列求和程序中的前几条 `li` 指令，并观察电路中GPR的状态是否与ISA的状态一致。

实现完整的sCPU

接下来考虑如何实现 `add` 指令。由于对所有指令来说，取指和更新PC这两个步骤的行为都一样，因此可以直接复用之前实现的取指逻辑。

对于译码，由于 `add` 指令的行为和 `li` 指令不同，因此有必要识别当前取出的是什么指令。为了实现这一点，需要检查指令的 `opcode` 字段，如果是 `00`，则是 `add` 指令，如果是 `10`，则是 `li` 指令。要识别输入的取值，最适合的电路就是译码器！由于 `opcode` 只有2位，我们可以使用一个2-4译码器，它输出的独热码可以指示当前指令属于何种指令，这样的译码器称为指令译码器。这组独热码通常作为控制信号，用于控制部分电路如何工作，从而支持不同指令的功能。

考虑完操作码之后，还需要考虑操作数。和 `li` 指令相比，`add` 指令除了需要写入 `rd`，还需要读出 `rs1` 和 `rs2` 作为源操作数，需要同时通过两个读端口和一个写端口访问GPR。因此，我们还需要为GPR添加两个读口，添加后，GPR模块应至少包含如下端口信号：

- 第1个读端口: `raddr1` (读地址), `rdata1` (读数据)
- 第2个读端口: `raddr2`, `rdata2`
- 写端口: `waddr` (写地址), `wdata` (写数据), `wen` (写使能), `clk` (时钟)

读出源操作数后，`add` 指令的行为是将两数相加，然后将加法结果写入 `rd` 寄存器。前者可以通过加法器实现，但后者写入GPR时，GPR的 `wdata` 端口已经被 `li` 指令的结果占用。考虑到指令 `opcode` 编码的唯一性，一条指令不可能既是 `li` 指令又是 `add` 指令，因此可以根据指令的类别对写入GPR的数据进行**选择**：如果当前指令是加法指令，就选择加法器的结果，否则选择 `li` 指令的立即数。可以通过多路选择器实现这样的选择功能，将写入GPR的两个数据来源作为多路选择器的数据端，然后让译码时独热码中合适的信号连接到多路选择器的选择端，来控制多路选择器按照指令类型选出正确的数据，即可实现上述功能。

添加add指令

根据上文，在sCPU中添加 `add` 指令. 实现后，尝试让sCPU继续执行数列求和程序中的几条 `add` 指令，并观察电路中GPR的状态是否与ISA的状态一致。

最后是 `bner0` 指令。为了识别 `bner0` 指令，我们可以复用指令译码器的功能。至于操作数，除了指令中的 `rs2` 和 `addr`，还有一个隐含的 `R[0]`。由于 `bner0` 指令中 `rs2` 字段的位置和 `add` 指令中 `rs2` 字段的位置一样，因此可以复用 `add` 指令中读出 `rs2` 寄存器的逻辑。但 `bner0` 还需要读出 `R[0]`，因此可以把 `0` 作为GPR的 `raddr1` 端口的输入。不过这个端口已经被 `add` 指令的 `rs1` 占用，但也同样可以通过多路选择器解决问题。

读出源操作数后，`bner0` 指令需要比较两数是否相等，这可以通过比较器来实现。若比较结果不相等，需要将PC更新为 `addr` 字段。换句话说，只有当前指令为 `bner0` 指令，且比较结果不相等，才将PC更新为 `addr` 字段，其余情况应将PC更新为PC加1。同样地，我们可以借助多路选择器对PC寄存器的输入端进行选择。

最后，`bner0` 指令不会写入GPR，因此需要将GPR的 `wen` 置为无效。

添加bner0指令

根据上文，在sCPU中添加 `bner0` 指令。实现后，尝试让sCPU执行完整的数列求和程序，如果你的实现正确，你应该能看到PC最终为 `7`，且在某GPR中存放求和结果 `55`。

和数列求和电路进行对比

在学习数字电路时，要求你通过寄存器和加法器，计算出 `1+2+...+10` 的结果。现在你用sCPU完成了同样的计算，尝试对比两个方案各有什么优点和缺点。

重新审视CPU

恭喜！这是你实现的第一个CPU！尽管这个sCPU非常简单，但也能从中窥探出一些可以帮助我们理解CPU的性质。

上文讨论如何添加指令时，都是先分析指令的预期行为，然后根据指令的行为在数据流动的方向上依次添加所需的部件。数据在CPU中流动的路径和路径上的相关部件，称为CPU的**数据通路**(data path)，在上文的讨论中，属于数据通路的部件有GPR，加法器，存储器，比较器等。

当多条指令的数据通路出现冲突时，通常需要引入一些额外的电路，来控制数据如何流动，使得每条指令都能完成符合ISA规范的操作。这些额外的电路属于CPU中的控制逻辑，其中决定控制逻辑行为的信号称为控制信号。事实上，设计数据通路和控制信号，正是CPU设计中最关键的两个步骤。

我们将前文涉及到的控制逻辑及其在不同指令执行时的预期行为整理如下：

指令	wdata的选择	wen	raddr1的选择	更新PC的选择
add	加法器的输出	1	指令的rs1	PC + 1
li	立即数	1	X	PC + 1
bner0	X	0	0	指令的addr

其中，`X` 表示无关项(don't care)，选择什么都可以。例如 `bner0` 指令不会写入GPR，因此无论 `wdata` 选择什么，都不影响sCPU的正确性。注意到这些选择与指令的类型相关，也即，指令的类型决定了控制逻辑的行为，从而决定了数据通路中数据的流向，最终完成指令约定的功能。

观察你在Logisim中设计的sCPU，如果把取出的指令作为边界，将sCPU划分为左右两侧，你应该能发现，左侧向右侧传递的信息只有指令。同时，右侧电路的控制逻辑最终都是由指令译码器所输出的独热码控制的，而指令译码器的输入端就是指令的操作码，因此可以说，右侧的电路完全受到指令的控制。这恰恰印证了上一节所说的：

在CPU上执行程序 = 用程序编译出的指令序列控制CPU电路进行状态转移

最后，我们通过一些必做题来加深对指令的理解。

计算10以内的奇数之和

编写一段指令序列，计算10以内的奇数之和，即 $1+3+5+7+9$ 。然后尝试用你设计的sCPU指令这段指令序列，检查运行结果是否符合预期。

完成后，你对计算机的"存储程序"思想有什么认识？

添加新指令

尝试为sISA添加一条新指令 `out rs`，执行该指令后，会将 `R[rs]` 以十六进制的形式输出到七段数码管。你可以自行决定这条指令的编码。

然后，在sCPU中实现 `out` 指令，并修改数列求和程序，使得在计算出结果后，能在七段数码管中显示计算结果。

能设计一条让10个数相加的指令吗?(思考题)

上述数列求和的程序需要执行数十条指令才能计算出结果。有同学建议，为了提升sCPU的计算效率，应该设计一条让10个数相加的指令，这样就可以通过一条指令直接计算

$1+2+\dots+10$ 。该建议是否合理？尝试从程序，ISA和CPU电路设计的角度给出你的理由。

RTFM

我们已经实现了sCPU，可以计算 $1+2+\dots+10$ 的和。虽然实现这个处理器确实让我们对处理器如何工作有更深入的认识，但从实用性的角度来看，sCPU由于各种限制，无法运行更复杂的程序。事实上，这些限制归根到底是因为sISA这个指令集过于简单，例如：

- PC寄存器的位宽只有4位，这意味着，程序最多只能包含16条指令。
- GPR的位宽只有8位，无法表示大于255的数据。
- 指令的功能有限，例如无法进行减法操作，更不用说乘法和除法。

而在下一周的学习中，我们将会搭建一个功能完备的RISC-V处理器，它可以运行更多程序，甚至有潜力运行超级玛丽游戏。这周作为铺垫，首先就先来了解RISC-V吧。

RISC-V是近十年流行起来的开放指令集架构，它采用模块化的思想，把指令划分成不同模块，除了基础指令集RV32I，还有各种指令扩展，包括乘除扩展，浮点扩展，原子操作扩展等。开发者可以根据自身需求选择一个或多个扩展，也可以一个扩展都不选，这种灵活性受到了开发者的喜爱。

作为一个真实的ISA，RISC-V规范的细节有相应的官方手册来描述。我们希望大家能养成阅读官方手册的好习惯，我们在**飞书群文件**中放了RISC-V的官方手册，虽然阅读英文版的原版手册效果更好，但我们仍然放了中文翻译后的手册来供大家选择，注意**翻译版在帮助用户理解方面有很大的帮助，但**

翻译过程中可能会有不准确或遗漏的情况。如果你是第一次接触ISA和处理器设计的相关知识，你可能会感到理解官方手册的每一处细节对你来说并不容易，不过我们将引导你从手册中寻找一些RV32I相关的关键信息。

通过RTFM初步了解RISC-V指令集

查阅RISC-V手册的目录，你发现RV32I在哪一章进行介绍？尝试在该章节中查阅RV32I的相关内容，回答下列问题：

1. PC寄存器的位宽是多少？
2. GPR共有多少个？每个GPR的位宽是多少？
3. `R[0]` 和sISA的 `R[0]` 有什么不同之处？
4. 指令编码的位宽是多少？指令有多少种基本格式？
5. 在指令的基本格式中，需要多少位来表示一个GPR？为什么？
6. `add` 指令的格式具体是什么？
7. 还有一种基础指令集称为RV32E，它和RV32I有什么不同？

将上述问题的答案也放入之前写的问题回答文档，放入作业提交的文件夹中。

作业提交

1.  提交你们用Logisim搭建的 `sCPU`，提交保存后的 `main.circ` 文件即可，并将这个文件复制到 `姓名-专业班级-Great-10` 文件夹中。
2. 所有必做题的回答所整理的**Markdown**文档也放在上述文件夹里。
3. **计算机组成原理部分与体系结构**也需要提交你的**Markdown**笔记（我们**推荐**大家进行**总结归纳和心得记录**，这真的很有帮助），也放在上述文件夹里并进行压缩。
4. 如果你学有余力完成了下面的拔高内容，则把文件夹重命名，格式为 `姓名-专业班级-NewStar-10`。
5. 将你的作业压缩为zip格式并提交到[作业提交表单](#)。

拔高内容

数据结构

想必有不少同学之前或多或少都做了不少lcthw的内容，你可能会产生这样的疑惑：“这玩意怎么这么难，这都是怎么想到的，学不会啊”，这是因为lcthw要求你的相关思想和基础都很过关，但是**缺**

少了数据结构——考研408之一，做lcthw很多内容会**极度痛苦**，同时不理解堆栈的相关概念将使得计组的学习举步维艰。

那么接下来的一部分路线就是：**学习数据结构，补足缺下的知识和思想，再去完成lcthw。**

数据结构

我们推荐下列方式进行数据结构的学习，可以选择适合自己的方式：

1、b站视频学习

https://www.bilibili.com/video/BV1tNpbekEht/?spm_id_from=333.337.search-card.all.click&vd_source=0ec8697614b7c28a28c8b55a23639096

2、可视化网站

https://www.hello-algo.com/chapter_array_and_linkedlist/linked_list/

3、书籍

这里推荐《大话数据结构》，是一本通过可视化描述来介绍抽象数据结构的书籍，如果你对数据结构不敏感，不妨看看其中的例子，飞书群文件中可以找到。

我们需要学习到有关**栈，堆，队列，链表**的所有知识，依旧是提交你的**Markdown**笔记。

 在这一部分学习过程中，我们会学到线性结构的所有知识，关于栈，堆，队列，链表等等；同时这些知识很快我们就会在lcthw中进行实践。

Learn C the hard way

这是“一生一芯”的**必须完成部分**如下：

Linux中的C语言进阶学习

在Linux中完成**Learn C the hard way**  练习0~22, 练习24~25, 练习27~33, 你需要将示例代码拷贝到Linux中编译并运行, 结合文字RTFSC理解示例代码, 并完成相应的附加题。

注意, 上述习题将是入学答辩的一部分. 如果你被某些题目卡住, 可以通过STFW或在预学习群中询问.

虽然这份教程的C语言知识点和之前的《Linux C编程一站式学习》差不多, 但它还包含了Makefile, Valgrind等各种工具的使用介绍, 以及一些值得学习的编程理念, 这些对大家将来进行开发都是很有帮助的, 非常值得大家在Linux上进行实践. 事实上, 如果你之前的C语言基础足够牢固, 你不应该在阅读代码上花费过多的时间; 相反, 如果你在阅读代码时遇到不少困难, 你应该感谢这次机会帮助你查漏补缺.

虽然一生一芯的讲义划定了学习的范围, 但想要技术很强的话, 我们建议都可以试着去学习。

Learn C the hard way

其中的**26、37-41、43、45-47不需要学习**, 性价比比较低, 不推荐学习, **其他内容我们都很推荐学习**, 想要技术很强的话, 都可以试着去学习, 并在其中锻炼自己gdb等debug工具使用和相关能力思维。

【Learn C the hard way】: <https://wizardforcel.gitbooks.io/lcthw/content/preface.html>

将你完成的所有练习放入一个名为 `lcthw` 的文件夹, 并将该文件夹放入作业提交文件夹中。

计算机组成原理

基础部分其实已经涉及到一点计算机组成原理的知识, 但大部分都是我们提取出大家需要马上实践的部分来学习, 仅仅是这个庞大科目的冰山一角, 如果你学有余力, 想要系统性的学习计组, 请参考下面, 但依旧需要完成基础部分的文档中的问题。

学习内容

记得随手typora记录笔记和继续自己的学习记录哦! (所有指令是不需要大家记忆的, 需要用到的时候查手册即可), 我们提供了2门视频课程, 请同学们结合自己的实际选择。

计算机组成原理/计算机组成与设计 (**CH1-CH2的内容学习**, 该课程体系完善, 效果好, 但理解起来较为困难)

王道计算机考研 计算机组成原理 (**第一章, 第二章的内容学习**, 其中**2.1.1~2.2.3涉及到数字电路部分可以选择跳过**, 该课程更为通俗易懂, 对于0基础学生更为友好)

两门课程的知识学习顺序不一样，但最终学习到的知识是一样的，依旧是提交你的 **Markdown** 笔记。

！ 注意

第一个视频课程是使用教材为**MIPS指令集版本**，我们基础部分讲述的与之后设计芯片，使用的是**RISC-V指令集版本**，但是考虑到该课程体系完善，效果好，指令集思想是相通的，因此我们仍然选择了该视频课程，配套书籍为黑皮书：计算机组成与设计：硬件软件接口，图示如下（**飞书群内有电子版资料**），想搭配书籍的同学可以参考——不推荐纯看书，黑皮书阅读难度大，知识点深而且广，很难理清知识点。

因此，**也可以选择更适合你的计组课程或者硬啃黑皮书**，学到知识就好



计算机组成与设计：硬件软件接口（原书第5版）

[戴维 A. 帕特森 \(David A. Patterson\)](#), [约翰 L. 亨尼斯 \(John L. Hennessy\)](#)

☆ 5.0 / 5.0 9 comments 心 书 同

《计算机组成与设计：硬件/软件接口》是计算机组成与设计的经典畅销教材，第5版经过全面更新，关注后PC时代发生在计算机体系结构领域的革命性变革——从单核处理器到多核微处理器，从串行到并行。本书特别关注移动计算和云计算，通过平板电脑、云体系结构以及ARM（移动计算设备）和x86（云计算）体系结构来探索和揭示这场技术变革。

与前几版一样，本书采用MIPS处理器讲解计算机硬件技术、汇编语言、计算机算术、流水线、存储器层次结构以及I/O等基本功能。

种类: [Computers](#) [C](#)
年: 2015
出版社: 机械工业出版社
页: 557
ISBN 13: 9787111504825
文件: PDF, 153.37 MB

Content Type: 书籍
出版: 5th
语言: Chinese
ISBN 10: 7111504828
系列: 计算机科学丛书
IPFS: [CID](#), [CID Blake2b](#)

▶ 在线阅读

📄 下载 (PDF, 153.37 MB)



发送到 [✉](#) [PB](#) [a](#)

平装 ▼

[书籍有问题吗?](#) ▼

? 难度这么高几乎无法完成，那拔高作业布置这么多是为什么？

1. 想让大家明白自己要学的内容还有很多。
2. 让大家可以自由选择拔高的学习内容，以上两项，皆可自由选择学习，不存在非常强烈的关联先后关系。
3. 并没有要求大家拔高作业全部完成，而是根据自己情况，可以完成多少就完成多少。

也请大家劳逸结合，考虑自己的能力和精力，合理学习，不要为了求快，去燃烧自己的热情与生命，这样是非常得不偿失的